



kubectl Commands in Kubernetes

Introduction

kubectl is the **command-line tool** for managing Kubernetes clusters.

- What is kubectl?
- Why is kubectl important?
- How do we use kubectl for Kubernetes management?

This guide will help you **understand essential kubectl commands** with practical examples.

Section 1: Learn

What is kubectl?

kubectl (Kubernetes Control) is a CLI tool used to **deploy, inspect, and manage** applications running in a Kubernetes cluster.

Feature	Description
Cluster Management	Create, delete, and update cluster resources
Pod and Deployment Control	Manage Pods, Deployments, and ReplicaSets
Networking	Expose services and monitor communication
Troubleshooting	Check logs, events, and system health



Why Use **kubectl**?

1. **Easy interaction with Kubernetes clusters** – Simplifies resource creation and management.
2. **Efficient application deployment** – Deploy, scale, and update applications smoothly.
3. **Better monitoring and troubleshooting** – Fetch logs, check system status, and resolve issues quickly.
4. **Automates repetitive tasks** – Execute batch operations with scripts.

Interesting Fact

The name **kubectl** is pronounced "**kube control**" or "**kube cuddle**", depending on user preference!

Section 2: Practice

1. Basic **kubectl** Commands

Check Cluster Information

```
kubectl cluster-info
```

View All Nodes in the Cluster

```
kubectl get nodes
```

List All Pods in a Namespace

```
kubectl get pods -n default
```



Describe a Specific Pod

```
kubectl describe pod <pod-name>
```

Delete a Pod

```
kubectl delete pod <pod-name>
```

2. Deployment & Scaling Commands

Create a Deployment

```
kubectl create deployment my-app --image=nginx
```

Scale a Deployment

```
kubectl scale deployment my-app --replicas=5
```

Check Deployment Status

```
kubectl rollout status deployment/my-app
```

Rollback a Deployment

```
kubectl rollout undo deployment/my-app
```

3. Networking Commands

Expose a Deployment as a Service

```
kubectl expose deployment my-app --type=NodePort --port=80
```



Get All Running Services

```
kubectl get services
```

Check Logs of a Running Pod

```
kubectl logs <pod-name>
```

Execute a Command Inside a Running Pod

```
kubectl exec -it <pod-name> -- /bin/sh
```

4. Debugging & Monitoring Commands

Get Resource Usage of Nodes

```
kubectl top nodes
```

Check Events for a Pod

```
kubectl get events --sort-by=.metadata.creationTimestamp
```

Watch Logs in Real Time

```
kubectl logs -f <pod-name>
```

Get Details About the API Server

```
kubectl version --short
```



Section 3: Know More

Frequently Asked Questions

1. What is the most commonly used **kubectl** command?
 - **kubectl get pods** is used frequently to check running applications.
2. How do I restart a Pod?
 - Kubernetes does not provide a restart command. Instead, delete the Pod, and the ReplicaSet will recreate it:

```
kubectl delete pod <pod-name>
```

3. How do I check failed Pods?
 - Use **kubectl get pods --field-selector=status.phase=Failed**.
4. What is the difference between **kubectl apply** and **kubectl create**?
 - **kubectl apply** is used for **updating existing resources**, while **kubectl create** is for **creating new resources**.
5. How do I delete all Pods in a namespace?
 - Run:

```
kubectl delete pods --all -n default
```

Conclusion

kubectl is the **primary tool** for interacting with Kubernetes clusters.

By mastering **kubectl** commands, users can **efficiently deploy, monitor, and troubleshoot applications**, ensuring **smooth Kubernetes operations**.